

Compiling a Hopfield Network into Clockless Logic

Aarav Vidhawan · Advised by Prof. Bill Lin

Department of Electrical and Computer Engineering, UC San Diego

Summer Research Internship Program (SRIP), 2026 · github.com/a-vidhawan/ECESRIP

Abstract

A Hopfield associative memory updates each neuron as $s_i = \text{sign}(\sum_j W_{ij} s_j)$. The conventional hardware realisation is a multiply–accumulate followed by a comparator. This project asks what happens if each neuron is instead compiled directly into **two-level Boolean logic**, and if the clock is then removed entirely, so that the network settles as a combinational feedback circuit.

Both halves turn out to be tractable, and both have a catch that only appears when measured.

The synthesis half works because an associative memory does not need to be correct everywhere — only on the states it actually visits. Specifying each neuron only over a bounded operating region and leaving the rest as don't-cares reduces the specification from 2^d rows to $M \cdot \sum_{k \leq h} C(d, k)$, which is polynomial in the fan-in where the full table is exponential. Measured: **31–54 product terms against 627–2,918** for the fully specified function at fan-in 16, and the method remains feasible at fan-in 32 where the complete table has 4.29×10^9 rows and cannot be enumerated at all.

The clockless half works because update order can be enforced by propagation delay rather than by a clock edge. Colouring the coupling graph and giving each colour class a different delay value prevents mutually coupled neurons from committing simultaneously. Two conditions on that mechanism were established by measurement and neither was obvious in advance: the invariant is on the delay **values** rather than on the colour labels, and the delay elements must be **inertial** rather than transport.

A complete network was built at $N = 256$ neurons and 12,357 product terms, and verified in RTL against its own behavioural model on 240 of 240 test vectors.

1. The problem

A Hopfield network is a recurrent network of binary threshold units whose symmetric weight matrix defines an energy function. Stored patterns sit at local minima; recall consists of initialising the network near a pattern and letting it descend to that minimum. The attraction for hardware is that this is a *settling* process rather than a computation — there is no instruction stream, and in principle no clock.

Two obstacles stand in the way of a direct logic implementation.

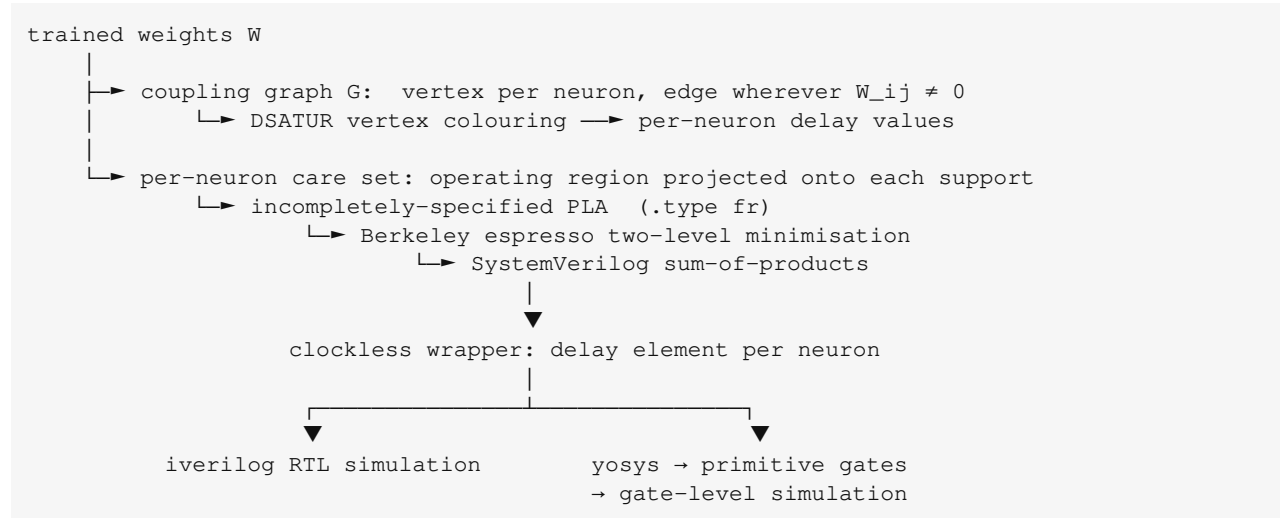
The truth table is exponential. A neuron with fan-in d has a 2^d -row truth table. At $d = 32$ that is 4.29×10^9 rows — not merely large to minimise, but impossible to *write down*. Existing LUT-based neural network work (NullaNet, LogicNets, PolyLUT) sidesteps this by sampling which input combinations a trained network actually produces, which gives an empirical frequency rather than a guarantee.

Removing the clock creates a correctness problem. Convergence guarantees for Hopfield networks assume serialised updates. If two mutually coupled neurons update simultaneously, each acts on the other's stale value, and the network can enter a limit cycle instead of settling. Measured exhaustively at N

= 16: **60.2%** of all 2^{16} states cycle under fully synchronous updating. Something has to impose an order, and in a clockless circuit there is no clock to impose it.

2. Approach

The pipeline compiles a trained weight matrix into synthesizable RTL:



Two design decisions carry most of the weight.

The care set is derived, not sampled. The operating region is defined as all states within Hamming distance h of a stored pattern. Its projection onto neuron i 's support is enumerated directly — the projection of a corrupted pattern is just that pattern's projection with at most h of *those* bits flipped — giving a closed-form size of $M \cdot \sum_{k \leq h} C(d, k)$. This is a guarantee over a stated radius rather than an observed activation frequency.

Order comes from delay, not from a clock. Each neuron's output passes through a delay element in its feedback path. Neurons in different colour classes get different delay values, so no two coupled neurons can commit at the same instant. There is no clock generator, no distribution network, and no sequencer.

3. Results

3.1 Don't-care synthesis dissolves the exponential wall

fan-in	full table rows	care rows	care %	product terms (don't-care)	product terms (full)
16	65,536	2,768	4.22%	31–54 (mean 40)	627–2,918 (mean 1,385)
24	16,777,216	9,300	0.055%	10–58 (mean 27)	infeasible to enumerate
32	4,294,967,296	21,956	0.001%	5–27 (mean 14)	infeasible to enumerate

The saving is in what must be **specified**, and only then in the term count that follows. At fan-in 48 espresso itself times out at 30 minutes — the limit is minimiser runtime, not term count.

What this costs. Off the operating region the function is unspecified, so espresso chooses freely and the minimised network diverges from the exact threshold network. Measured agreement on uniformly random

states is **2.5%**. Inside the operating region the two are indistinguishable, including at Hamming distance 5 — beyond the radius-3 care set they were built from. This is the method's price, not a defect to conceal: it is fine for recall and unsuitable for adversarially chosen inputs.

Because the network is recurrent, this is a stronger claim than it would be in a feed-forward network. An unspecified input that is later realised can create a fixed point the target function does not have. Behavioural equivalence had to be verified, not assumed.

3.2 The scheduling invariant is on delay values

Partitioning by parity of neuron index — the obvious first scheme — fails. On an $N = 16$ network with 43 coupling edges at 35.8% density, the chromatic number is **6**, and parity leaves **19 of 43 coupled pairs (44.2%)** inside a common class. The highest-degree neuron shares a class with six of its eleven neighbours.

The 2×2 that settled the question:

	distinct delay values	identical delay values
proper colouring	100% settled	0% settled
parity (no colouring)	0% settled	—

The upper-right cell is the result. A *valid proper colouring* whose classes are all assigned the same delay value satisfies every graph-theoretic requirement of the partition and settles in none of the trials. The invariant is on delay values, not on class labels — a distinction with no counterpart in a clocked implementation, where a colour simply *is* a phase.

The project's own verifier had this bug and would have passed the 0% schedule.

The particular values are immaterial. Five delay families — consecutive integer multiples, powers of two, prime multiples, mutually coprime offsets, and an irrational ratio — all reach a fixed point from 100% of random initial states at $N = 16, 32$ and 64 . Twelve random permutations of the same six primes over the same six classes give a standard deviation of 0.00. Distinctness is the whole requirement.

3.3 The delay element must be inertial

This was the sharpest result of the project, and it arrived by chasing an anomaly: a zero-delay *reference* model settled less often than the glitchy gate-level designs it was supposed to be a reference for, which is impossible if the schedule works.

An **inertial** delay cancels a pending transition when its cause reverts before the delay elapses. A **transport** delay queues every transition and delivers all of them. Because each neuron evaluates continuously rather than once per pass, a neuron's target can revert after a transition is scheduled and before it commits — and a transport element then commits that superseded value onto neighbours whose states have already changed.

Identical networks, identical partitions, identical delay values, identical initial states; only the delay semantics differ:

N	inertial	transport
16	100%	0.7%
32	100%	15.7%
64	100%	14.7%

128	100%	7.0%
-----	------	------

Commensurate delay values realigning was ruled out first, which is what made the delay semantics the only remaining explanation.

This is not a hazard mitigation that can be added or omitted. It is a requirement of the scheme — and notably, a phase-shifted clock cannot supply it, because a clocked node samples at an edge and never observes a transition that appeared and disappeared between two edges.

3.4 End-to-end verification at $N = 256$

$N = 256$, $M = 4$, fan-in 16, care radius 3, $\chi = 4$, zero delay-value conflicts. 12,357 product terms across 256 neurons, emitted as SystemVerilog and simulated in Icarus Verilog.

HD	n	RTL settled	RTL recall	RTL = simulator
0	60	100.0 [94.0–100.0]	100.0 [94.0–100.0]	100.0
1	60	100.0 [94.0–100.0]	100.0 [94.0–100.0]	100.0
3	60	100.0 [94.0–100.0]	100.0 [94.0–100.0]	100.0
5	60	100.0 [94.0–100.0]	100.0 [94.0–100.0]	100.0

240 of 240 vectors. The last column is the one that matters: it is what licenses using the faster event-driven simulator for larger networks at all — and it licenses it at $N = 256$, not at $N = 4096$.

3.5 Capacity, robustness, optimisation

Capacity. Margin-based retraining (Krauth–Mertens minover under a sparsity mask, with the margin κ chosen adaptively as the largest feasible value) stores all patterns with $\geq 95\%$ recall at a loading of $\alpha = M/N = 0.5$, against **0.138** for the classical outer-product rule. Corruption tolerance improves with size: $\geq 90\%$ recall out to 5% of bits corrupted at $N = 64$, 16% at $N = 128$, and **19% at $N = 256$** .

Process variation. Independent random perturbation of each neuron's delay value leaves settling at 100% out to $\pm 348\%$ spread at 3σ . More interestingly, applying variation to a *degenerate* schedule — all classes assigned the same nominal delay — raises settling from 67% to 100%, because continuous variation makes nominally equal delays distinct with probability one. The practical corollary is that silicon variation helps; the hazard is delays made equal *by construction*, as an identical-buffer-chain layout would give.

Optimisation. Mapping MAX-CUT onto the same machine by setting $w = -A$, the network reaches a fixed point on 100% of instances and attains **99.5%** of the best cut weight found by any method tested, at roughly **100×** lower simulated cost than single-flip simulated annealing at equal restarts. A fixed point is a locally optimal cut; no asymptotic claim is made.

4. Negative results and self-corrections

Every headline number is regenerated from source by `audit_claims.py`. Final status: **14 verified, 2 revised, 2 retracted**.

Retracted outright. (i) "32 universal oscillators never converge under any configuration" — false; every graph-coloured schedule settles all 32 across 18 independent schemes, and the original claim generalised from having tried only parity variants. (ii) "Noise is a third delay mode" — the noise RTL emitted byte-

identical delays to depth mode, because $\text{round}(d + \mathcal{U}(-0.5, 0.5))$ almost always returns d . Every three-way mode comparison in the project was really two-way.

Revised. An early estimate claimed the LUT approach was $2.4\text{--}2.8\times$ smaller than threshold gates. Synthesis says **$1.52\times$ smaller on an ASIC proxy and $1.19\times$ larger on FPGA** — 4-bit weights suffice where the estimate assumed 8, and 6-LUT packing favours the adder tree. *Do not lead with area.*

A baseline that beats us. At $N = 64$, $M = 4$ a nearest-match content-addressable memory is **2,858 gates to our 7,020**, and recalls 100% at every Hamming distance where this design manages $\sim 57\%$. The verdict is radius-dependent — at care radius 2 we are $2.1\times$ smaller — but the loading sweep is unambiguous: 4/4 patterns stored at $M = 4$, 6/8 at $M = 8$, 0/16 at $M = 16$. This design fails on *storage* before it fails on *area*.

Where the failures actually are. Decomposing recall failures on random initial states: spurious convergence accounts for $\sim 75\%$, oscillation for $\sim 2\%$. Scheduling addresses the oscillation column. Only loading addresses the other.

5. Position relative to prior art

The honest summary is that the scheduling mechanism is not novel, and establishing that was itself a substantial part of the work.

Colour-partitioned update ordering for recurrent stochastic networks traces to **Geman & Geman (1984)**. **Gonzalez et al. (AISTATS 2011)** built the Chromatic Gibbs sampler on the same energy argument. In hardware and in this exact field, **Aadit et al. (Nature Electronics, 2022)** describe their FPGA Ising machine as "*a low level hardware-level implementation of chromatic Gibbs sampling*", and their bibliography collects further colour-block hardware — an FPGA parallel Gibbs accelerator (FlexGibbs, FCCM 2019), FPGA Ising annealing processors (Yoshimura et al., 2016, 2017), and an FPGA restricted Boltzmann machine (Patel et al., 2020).

That same paper also states the sizing constraint — "*the MAC must finish its computation before the next color block is updated*" — and separately reports that deliberately violating it ("overclocking", connected to Hogwild!-Gibbs) *improves* time-to-solution.

Two things survived a full-text review of the closest references. The specific combination of colour-partitioned ordering **and** no periodic timing reference was not found. Neither was the inertial-cancellation requirement of §3.3 — and that one is a structural incapability of the art rather than an oversight, since every implementation above samples at a clock edge.

The operating-region don't-care derivation is also untouched by that art, which derives don't-cares by sampling observed activations rather than in closed form from a bounded region.

6. Method

Results carry an evidence tier: **T1** measured in RTL, **T2** measured by a tool (yosys, espresso), **T3** from the event-driven simulator, **T4** analytical estimate. Proportions are reported as Wilson 95% confidence intervals rather than the normal approximation, because many estimates sit at exactly 0 or 1 where the normal interval collapses to zero width and overstates certainty.

The audit script exists because three of the project's own conclusions were overturned by later

measurement. The intent was that the record should show that rather than hide it.

7. Limitations

- **No silicon and no PDK timing.** Area figures are yosys cell counts after technology-independent mapping.
 - **RTL verification stops at $N = 256$.** Larger results come from the event-driven simulator, validated against RTL at $N = 16$ and $N = 256$ and not beyond.
 - **Random bipolar patterns only.** No correlated or real data, which would change basin geometry.
 - **Off-region behaviour is unspecified by construction** — 2.5% agreement with the exact network on uniformly random states.
 - **The gate-level hazard comparison is incomplete.** Inertial delay was re-measured on a corrected testbench; the dual-rail/C-element comparison was not, and its numbers come from a testbench with a known premature-readout bug.
-

8. Reproducing

Requires `python3` (`numpy`, `matplotlib`), `iverilog`, `yosys`, and Berkeley `espresso` on `PATH`.

```
python3 phase2/paper/audit_claims.py           # regenerate every headline number
python3 phase2/clockless/rtl_n256.py --N 256 --M 4 --degree 16 --radius 3
python3 phase2/clockless/analyze_coupling.py    # colouring vs parity
python3 phase2/phase10_glitch/inertial_required.py # inertial vs transport
```

Repository: github.com/a-vidhawan/ECESRIP

References

1. J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. *PNAS* 79(8):2554–2558, 1982.
2. S. Geman and D. Geman. Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images. *IEEE TPAMI PAMI*-6(6):721–741, 1984.
3. W. Krauth and M. Mézard. Learning algorithms with optimal stability in neural networks. *J. Phys. A* 20(11):L745, 1987.
4. J. Gonzalez, Y. Low, A. Gretton, C. Guestrin. Parallel Gibbs sampling: from colored fields to thin junction trees. *AISTATS*, PMLR 15:324–332, 2011.
5. N. A. Aadit et al. Massively parallel probabilistic computing with sparse Ising machines. *Nature Electronics* 5:460–468, 2022.
6. S. Nikhar, S. Kannan, N. A. Aadit, S. Chowdhury, K. Y. Camsari. All-to-all reconfigurability with sparse and higher-order Ising machines. *Nature Communications* 15:8977, 2024.
7. G. G. Ko, Y. Chai, R. A. Rutenbar, D. Brooks, G.-Y. Wei. FlexGibbs: reconfigurable parallel Gibbs sampling accelerator for structured graphs. *FCCM*, 2019.
8. Y. Umuroglu, Y. Akhauri, N. J. Fraser, M. Blott. LogicNets: co-designed neural networks and circuits

for extreme-throughput applications. *FPL*, 2020.

9. S. H. Unger. *Asynchronous Sequential Switching Circuits*. Wiley-Interscience, 1969.
10. R. K. Brayton, G. D. Hachtel, C. McMullen, A. Sangiovanni-Vincentelli. *Logic Minimization Algorithms for VLSI Synthesis*. Kluwer, 1984.